Mission 5 – Haute disponibilité de GLPI

10 février 2026

Table des matières

1 Objectif de la mission **2**

2 Mise en place du load balancing **2**
2.1 Installation de HAProxy 2
2.2 Configuration du load balancer 2

3 Haute disponibilité du load balancer **3**
3.1 Installation de Keepalived 3
3.2 Configuration VRRP 3
3.3 Test de bascule 3

4 Réplication de la base de données **3**
4.1 Configuration du serveur master 4
4.2 Création utilisateur de réplication 4
4.3 Configuration du slave 4

5 Synchronisation des fichiers GLPI **4**
5.1 Installation serveur NFS 4
5.2 Montage côté client 5

6 Tests de validation **5**

7 Conclusion **5**

Objectif de la mission

L'objectif de cette mission est de garantir la continuité de service de GLPI même en cas de panne d'un serveur. Pour cela, nous mettons en place une architecture redondante basée sur un load balancer, une réplication de base de données et une synchronisation des fichiers.

L'idée principale est simple : aucun composant ne doit être unique ou critique. Chaque service doit pouvoir être repris automatiquement par un autre en cas de panne.

Mise en place du load balancing

Le premier élément mis en place est un répartiteur de charge basé sur HAProxy. Son rôle est de recevoir toutes les connexions des utilisateurs et de les envoyer vers un serveur web disponible.

2.1 Installation de HAProxy

Sur la machine `haproxy01`, nous installons le service avec la commande suivante :

```
apt update
apt install haproxy -y
```

La commande `apt update` permet de mettre à jour la liste des paquets disponibles, tandis que `apt install` installe le logiciel HAProxy.

2.2 Configuration du load balancer

Le fichier de configuration principal se trouve ici :

```
/etc/haproxy/haproxy.cfg
```

Nous définissons un frontend qui écoute les connexions utilisateurs :

```
frontend glpi_front
    bind *:80
    bind *:443 ssl crt /etc/ssl/certs/glpi.pem
    redirect scheme https if !{ ssl_fc }
    default_backend glpi_back
```

Cette configuration permet de forcer le HTTPS et de rediriger tout le trafic vers le backend.

Le backend contient les serveurs web :

```
backend glpi_back
    balance roundrobin
    server web01 10.54.0.11:80 check
    server web02 10.54.0.12:80 check
```

Le mode `roundrobin` permet de répartir les connexions de manière équilibrée entre les serveurs.

Après modification, nous redémarrons le service :

```
systemctl restart haproxy
systemctl enable haproxy
```

La première commande redémarre le service, la deuxième permet son lancement automatique au démarrage.

Haute disponibilité du load balancer

Afin d'éviter que HAProxy devienne un point unique de défaillance, nous ajoutons une seconde machine `haproxy02`.

3.1 Installation de Keepalived

Keepalived permet de gérer une IP virtuelle (VIP). Nous l'installons avec :

```
apt install keepalived -y
```

3.2 Configuration VRRP

Sur `haproxy01`, nous définissons une priorité élevée :

```
vrrp_instance VI_1 {
    state MASTER
    interface eth1
    virtual_router_id 51
    priority 100
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass crash123
    }
    virtual_ipaddress {
        10.54.0.100
    }
}
```

Sur `haproxy02`, la priorité est plus faible, ce qui permet la bascule automatique.

3.3 Test de bascule

Nous simulons une panne avec :

```
systemctl stop haproxy
```

Cette commande arrête HAProxy volontairement. Nous vérifions ensuite que la VIP est reprise par le second serveur avec :

```
ip a
```

La commande `ip a` permet de vérifier les adresses IP actives sur la machine.

Réplication de la base de données

La base de données est un élément critique, car elle contient toutes les données GLPI.

4.1 Configuration du serveur master

Sur `glpi-db01`, nous activons la réplication :

```
nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Nous ajoutons :

```
[mysqld]
server-id = 1
log_bin = /var/log/mysql/mysql-bin.log
```

Puis nous redémarrons MySQL :

```
systemctl restart mysql
```

4.2 Création utilisateur de réplication

Dans MySQL :

```
CREATE USER 'replica'@'%' IDENTIFIED BY 'password';
GRANT REPLICATION SLAVE ON *.* TO 'replica'@'%';
FLUSH PRIVILEGES;
```

Cette commande crée un utilisateur dédié à la réplication.

4.3 Configuration du slave

Sur `glpi-db02` :

```
CHANGE MASTER TO
MASTER_HOST='10.54.0.21',
MASTER_USER='replica',
MASTER_PASSWORD='password';
```

Puis :

```
START SLAVE;
```

La commande `START SLAVE` lance la réplication.
Nous vérifions avec :

```
SHOW SLAVE STATUS\G
```

Les champs `Slave_IO_Running` et `Slave_SQL_Running` doivent être à YES.

Synchronisation des fichiers GLPI

Les fichiers GLPI doivent être accessibles depuis tous les serveurs web.
Nous utilisons ici NFS.

5.1 Installation serveur NFS

```
apt install nfs-kernel-server -y
```

Nous exportons le dossier GLPI :

```
/var/www/html/glpi/files 10.54.0.0/24(rw,sync,no_root_squash)
```

Puis :

```
exportfs -ra
```

La commande `exportfs -ra` applique la configuration.

5.2 Montage côté client

Sur chaque serveur web :

```
apt install nfs-common -y
mount 10.54.0.20:/var/www/html/glpi/files /var/www/html/glpi/files
```

Tests de validation

Nous vérifions le bon fonctionnement de l'architecture.
Pour tester le load balancing, nous utilisons :

```
curl http://10.54.0.100
```

Cette commande envoie une requête HTTP vers la VIP.
Pour tester la base de données :

```
mysql -u root -p -e "SHOW DATABASES;"
```

Cette commande affiche les bases de données disponibles.
Pour tester la haute disponibilité, nous simulons une panne :

```
systemctl stop haproxy
```

Le service doit rester accessible malgré tout.

Conclusion

Cette mission permet de transformer une architecture simple en une infrastructure résiliente. Grâce au load balancing, à la réplication de base de données et à la synchronisation des fichiers, le service GLPI devient capable de supporter des pannes sans interruption visible pour l'utilisateur.

Cette approche correspond aux standards utilisés dans les environnements de production modernes.